

Contents

CST-339 Lab 4: User Registration, Login, and Administration 2

 Thymeleaf Forms, PostgreSQL, and the User-Management Domain 2

 Overview 2

 Purpose 2

 Lab Outcomes..... 2

 Prerequisites..... 2

 Architectural Scope 2

 Thymeleaf and Dynamically Generated Web Pages 3

 Security Limitations 3

 GitHub Workflow 3

 Branch 1: Add Thymeleaf and Bootstrap 4

 Branch 2: Create the Application User Entity and Database Table 4

 Branch 3: Create the User Repository 6

 Branch 4: Create the User Service and ViewModels 6

 Branch 5: Create the Registration Form 7

 Branch 6: Create the Login Form and Manual Credential Check 8

 Branch 7: Create User Administration Pages 8

 Branch 8: Add the Responsive Navigation Bar 9

 Verification 9

 Deliverables..... 9

 1. GitHub Repository 9

 2. Video Demonstration 10

 3. AI References..... 10

Study Guide - Lab 4 11

 Dynamically Generated Web Pages 11

 MVC Request Processing 11

 Thymeleaf..... 11

 ViewModels..... 11

 Thymeleaf Binding..... 12

ViewModels.....Error! Bookmark not defined.

 Login versus Authentication..... 12

CST-339 Lab 4: User Registration, Login, and Administration

Thymeleaf Forms, PostgreSQL, and the User-Management Domain

Overview

In Labs 1-3, you built, persisted, tested, containerized, and deployed a professional Spring Boot REST API. Lab 4 adds application users and Lab 5 secures the application.

This separates user management from security enforcement. You will first build the underlying database model, repository, service, Thymeleaf forms, login workflow, and administrative pages. Only after that work is understandable and functioning will Lab 5 add Spring Security, BCrypt password encoding, authenticated sessions, and role-based authorization.

Important security boundary: Lab 4 is intentionally insecure. Passwords are temporarily stored and compared as plain text. Roles are stored as data but are not enforced as authorities. The login page demonstrates credential lookup, but it does not securely protect routes. Do not reuse this Lab 4 implementation in a production application. Lab 5 replaces these temporary mechanisms with Spring Security and BCrypt. Lab 5 demonstrates a secure application.

Purpose

This lab introduces Spring MVC and Thymeleaf as a narrowly scoped administrative interface inside an API-centered application. The Album and reporting features remain REST APIs. Thymeleaf is used only where browser forms are practical: user registration, login, initial password change, and user administration.

Lab Outcomes

- Add Thymeleaf and Bootstrap to an existing Spring Boot application.
- Create and persist an application-user entity in PostgreSQL.
- Use Spring Data JPA to save, retrieve, and update users.
- Use View Models rather than binding Thymeleaf forms directly to JPA entities.
- Create registration and login forms with server-side validation.
- Demonstrate manual credential lookup before Spring Security is introduced.
- Create a user-administration page for roles, enabled status, and password-change status.

Prerequisites

- Complete Labs 1-3.
- Begin with the existing Lab 2/Lab 3 Spring Boot application and PostgreSQL database.
- Confirm the application starts and the existing REST API works before changing the project.
- Use the existing feature-branch and pull-request workflow.

Architectural Scope

After finishing this lab, it will support two presentation styles:

Application Area	Presentation Style	Purpose
------------------	--------------------	---------

Album and reporting features	REST controllers returning JSON	API clients, Swagger, and Postman
User registration and administration	Spring MVC controllers with Thymeleaf templates	Browser forms and administrative workflow

This lab is not a return to a traditional MVC web-application course. Thymeleaf is a small supporting interface. The main application remains API-centered.

Thymeleaf and Dynamically Generated Web Pages

Thymeleaf belongs to the long tradition of server-side, dynamically generated web pages. Early web applications commonly produced HTML on the server using technologies such as CGI, PHP, JavaServer Pages, ASP, and later template systems associated with frameworks like Ruby on Rails, Django, and ASP.NET MVC. In this model, the browser requests a page, the server retrieves and processes data, and a template combines that data with HTML before sending the completed page to the client. Thymeleaf performs this role in the Spring ecosystem: a Spring MVC controller places data into a model, and a Thymeleaf template renders that model as HTML. In C# III, Razor pages fill this role.

Thymeleaf is best-of-breed and is especially well suited to administrative screens, forms, dashboards, and other pages where server-side rendering is simpler than building a separate JavaScript front end. It is not a replacement for React, Angular, or other client-side frameworks; it represents a different architectural choice. Modern applications often use both approaches: REST APIs for external clients and rich JavaScript applications, while using server-rendered Thymeleaf pages for internal tools, login screens, and user administration. In this lab, Thymeleaf provides a small, practical user interface without shifting the course away from its primary focus on Spring Boot APIs and security.

Security Limitations

- The password field contains plain text.
- The login controller compares submitted credentials in application code.
- No Spring Security filter chain is active.
- A user can know a role without the application securely enforcing that role.
- An enabled flag can be checked by the login workflow, but it does not protect every request.
- The first-password-change workflow can redirect a cooperative user, but it cannot securely prevent direct URL access until Lab 5.
- Lab 5 will hash passwords, authenticate users, establish secure sessions, enforce roles, and protect routes.

GitHub Workflow

Complete each branch in order. Each branch should contain one coherent feature and should be merged before the next branch begins.

1. Create the named feature branch.
2. Complete and test the required work.
3. Commit with a meaningful message.
4. Push the branch to GitHub.
5. Open a pull request and review the changes.
6. Merge the pull request into the working branch used for the lab.

Branch 1: Add Thymeleaf and Bootstrap

Add a small server-rendered interface to the existing API application. Bootstrap is used so that students can focus on Spring MVC, Thymeleaf, JPA, and the future Spring Security work rather than spending lab time on custom CSS.

Learning Expectations

- Difference between a REST controller and an MVC controller.
- How a controller returns a template name.

Required Work

- Add the Thymeleaf starter dependency. Do not add Spring Security yet.
- Add Bootstrap 5 through a CDN link in the shared page structure or in each initial template.
- Keep REST controllers and MVC controllers in separate feature packages.

Checkpoint

- Bootstrap styling loads in the browser.
- Existing JSON endpoints continue to work.

Commit and push the branch. Open a pull request, review the change, and merge it before beginning the next branch.

Branch 2: Create the Application User Entity and Database Table

Create the persistent user model that will support registration, login, administration, and later Spring Security authentication.

Learning Expectations

- JPA entity mapping.
- Unique constraints.
- Why the application should not name the table user.
- Why the password field is temporary and will change meaning in Lab 5.

Required Work

- Create a User entity in a user feature package.
- Use a generated primary key.
- Make username unique and required.
- Add password, role, enabled, and passwordChangeRequired fields.
 - The role property can have two values:
 - ADMIN
 - USER
- Update the database schema by adding a *users* table and verify the resulting table in pgAdmin.
 - The fields of the *users* table must match the entity's properties in name and type.
 - Seed the database with a user with the following attributes:

• username:	admin
• password:	password
• role:	ADMIN
• enabled:	true
• passwordChangeRequired:	true

Use this *users.sql* script:

```

CREATE TABLE IF NOT EXISTS users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL DEFAULT 'USER',
    enabled BOOLEAN NOT NULL DEFAULT TRUE,
    password_change_required BOOLEAN NOT NULL DEFAULT TRUE
);

COMMENT ON TABLE users IS 'Stores application users for registration and authentication.';
COMMENT ON COLUMN users.user_id IS 'Primary key for the user record.';
COMMENT ON COLUMN users.username IS 'Unique username used for sign-in.';
COMMENT ON COLUMN users.password IS 'Stored password hash.';
COMMENT ON COLUMN users.role IS 'User role such as USER or ADMIN.';
COMMENT ON COLUMN users.enabled IS 'Whether the account is enabled.';
COMMENT ON COLUMN users.password_change_required IS 'Whether the user must change their password at next sign-in.';

INSERT INTO users (username, password, role, enabled, password_change_required)
VALUES ('admin', 'password', 'ADMIN', TRUE, TRUE)
ON CONFLICT (username) DO NOTHING;

```

After running the script, use pgAdmin to verify the table and contents with a SELECT statement.

- In your source tree create a `sql_scripts` folder under your source's root. Save the `users` table script in that folder as `users.sql`. It is common professional pattern to store SQL scripts with the source code.
- Checkpoint
 - The entity compiles and maps correctly.
 - The `users` table exists in PostgreSQL.
 - The username column prevents duplicate values.
 - Boolean fields have appropriate defaults.
 - The existing Chinook tables remain intact.

Commit and push the branch. Open a pull request, review the change, and merge it before beginning the next branch.

Required Fields

Field	Purpose	Suggested Constraint	Lab 5 Change
userId	Database identity	Generated primary key	No conceptual change
username	Login name	Required and unique	Used by UserDetailsService
password	Temporary plain-text credential	Required	Stored as a BCrypt hash
role	Requested/assigned access level	Required; default USER	Converted to a granted authority
enabled	Account availability	Default true	Enforced by Spring Security
passwordChangeRequired	Temporary-password indicator	Default false	Enforced after authentication

Branch 3: Create the User Repository

Add the Spring Data JPA repository methods needed by registration, login, administration, and bootstrap initialization.

Learning Expectations

- Derived query methods.
- Optional results.
- Difference between `count()==0` and checking specifically for an administrator.

Required Work

- Create `UserRepository` extending `JpaRepository`.
- Add a username lookup method that returns `Optional<User>`.
- Add an existence check for duplicate usernames.
- Add an administrator lookup or role query needed by the bootstrap initializer.
- Write a focused repository test or verify the methods through the service test used in the next branch.

Checkpoint

- Users can be saved and retrieved.
- A user can be located by username.
- Duplicate usernames can be detected before save.
- The application can determine whether an administrator already exists.

Commit and push the branch. Open a pull request, review the change, and merge it before beginning the next branch.

Branch 4: Create the User Service and ViewModels

Place user-management rules in a service rather than in Thymeleaf controllers. Use ViewModels so that browser input is not bound directly to the JPA entity. ViewModels are DTOs used in the context of the View. DTOs are used in Clean Architecture to decouple application layers. Thus, ViewModels are best considered as Presentation DTOs.

Learning Expectations

- ViewModels versus entities.
- Service-layer validation.

- Why security-sensitive logic should not be scattered across controllers.

Required Work

- Create a registration DTO with username, password, and confirmPassword.
- Create a login DTO with username and password.
- Create a password-change DTO with currentPassword, newPassword, and confirmPassword.
- Create an administration DTO or command object for role and enabled-status updates.
- Create a UserService that supports registration, username lookup, login validation, user listing, account updates, and password replacement.
- Keep controllers responsible for HTTP/form flow and keep business rules in the service.

Checkpoint

- Controllers do not call the repository directly.
- JPA entities are not used as form-backing objects.
- Password confirmation is checked in the service or validation layer.
- Duplicate username rules are enforced consistently.
- User-management service tests pass.

Commit and push the branch. Open a pull request, review the change, and merge it before beginning the next branch.

Branch 5: Create the Registration Form

Create a public registration workflow. New self-registered users receive the least-privileged role USER. They must not be allowed to select an administrative role.

Learning Expectations

- Thymeleaf form binding.
- POST/Redirect/GET.
- Server-side validation.
- Least privilege for self-registration.

Do not present plain-text storage as acceptable practice. The temporary database value exists only so Lab 5 can replace it with BCrypt encoding.

Required Work

- Create GET and POST mappings for the registration page.
- Create a Thymeleaf form using th:object and th:field.
- Collect username, password, and confirm password.
- Validate required fields, matching passwords, and duplicate usernames.
- Save the user through UserService and redirect to the login page with a success message.

Checkpoint

- A new user can register through the browser.
- Mismatched passwords display a validation message.
- Duplicate usernames are rejected.
- The new record appears in PostgreSQL.
- The stored role is USER even if the browser request is manipulated.

Commit and push the branch. Open a pull request, review the change, and merge it before beginning the next branch.

Branch 6: Create the Login Form and Manual Credential Check

Create the login screen and prove that the application can retrieve a user and compare submitted credentials. This is a temporary credential-check demonstration, not secure authentication.

Learning Expectations

- Credential lookup versus authentication.
- Why a login form alone does not protect an application.

The Lab 4 login controller may demonstrate navigation state, but it must not be described as secure session management. Spring Security will own authentication and session handling in Lab 5.

Required Work

- Create GET and POST mappings for login.
- Create a Thymeleaf login form for username and password.
- Use UserService to locate the account, verify enabled status, and compare the submitted password with the stored password.
- Display one general error message for an invalid username or password rather than revealing which value failed.
- On a successful credential check, route the user according to passwordChangeRequired.
- Do not build a large custom authentication framework or custom security filter. Lab 5 will replace this controller logic.

Checkpoint

- Valid credentials produce a successful result.
- Invalid credentials display a general error.
- Disabled users are refused by the login workflow.
- A user with passwordChangeRequired=true is redirected to the password-change page.
- Students can explain why direct URLs remain unprotected in Lab 4.

Commit and push the branch. Open a pull request, review the change, and merge it before beginning the next branch.

Branch 7: Create User Administration Pages

Create a browser-based administrative interface for viewing and updating user accounts. The page exists in Lab 4, but its URL is not securely protected until Lab 5.

Learning Expectations

- Administrative CRUD through MVC forms.
- Why hidden fields and disabled controls are not security controls.
- Protection against locking all administrators out.

Required Work

- Create a Thymeleaf table that lists username, role, enabled status, and password-change status.
- Never render the password field.
- Create forms that allow role changes and enabled/disabled changes.
- Allow an administrator to mark a user as requiring a password change or initiate a password reset workflow.
- Validate allowed roles in server-side code.
- Prevent accidental disabling of the only administrator, or document and handle the recovery approach.

Checkpoint

- Users are listed from PostgreSQL.
- Password values are never displayed in HTML.
- Role and enabled-status changes persist.
- Invalid role values are rejected.
- The only administrator cannot be removed or disabled without a documented recovery path.
- Students can explain that Lab 4 page visibility is not authorization.

Commit and push the branch. Open a pull request, review the change, and merge it before beginning the next branch.

Branch 8: Add the Responsive Navigation Bar

Purpose

In this branch, you will add a shared Bootstrap navigation bar.

The navigation bar will provide consistent access to the application's user-management features:

- Login
- Register
- User Administration

The navigation bar must be responsive so that it works correctly on both desktop and mobile-sized screens.

Verification

1. Start the application without the Spring Security dependency.
2. Open the Thymeleaf landing page.
3. Register a new user and verify USER in PostgreSQL.
4. Try a duplicate username and show the validation result.
5. Complete a valid and invalid login credential check.
6. Demonstrate that a disabled account fails the login workflow.
7. Open the administration page and update a role and enabled status.
8. Restart with no administrator present and verify bootstrap creation.
9. Log in with the temporary administrator and complete the password-change workflow.
10. Demonstrate that a direct administration URL is still reachable without real authorization and explain that Lab 5 will correct this vulnerability.

Deliverables

1. GitHub Repository

- All nine feature branches or an instructor-approved equivalent branch structure.
- Pull requests and merge history.
- Meaningful commit messages.

- Updated README instructions for environment variables and local startup.
- No committed real passwords, secrets, or production credentials.

2. Video Demonstration

- Thymeleaf pages with Bootstrap styling.
- User registration and validation.
- PostgreSQL user record.
- Valid and invalid manual login checks.
- User administration without displaying passwords.
- Bootstrap administrator creation.
- Initial password change.
- An explicit explanation that Lab 4 is intentionally insecure and that Spring Security is not yet enforcing authentication or authorization.

3. AI References

- Prompt or prompts used.
- AI system and model.
- Date used.
- Summary of how the response influenced the work.
- Explanation of what the student reviewed, corrected, or rejected.

Study Guide - Lab 4

This expanded study guide preserves the original concepts while adding architectural emphasis on dynamic page generation, MVC request processing, ViewModels, and Thymeleaf.

Dynamically Generated Web Pages

Students should understand the basic architecture of server-side web applications.

1. What a dynamically generated web page is.
2. How dynamic pages differ from static HTML pages.
3. Why server-side rendering has been used since the earliest web applications.
4. Why dynamically generated pages can be viewed as an HTML template containing placeholders that are replaced with application data before the completed page is sent to the browser.
5. The historical evolution of server-side page generation, including CGI, PHP, JavaServer Pages (JSP), ASP, Razor Pages, and Thymeleaf.

MVC Request Processing

Students should understand the complete request lifecycle.

1. Browser requests a page.
 2. Spring MVC selects a controller.
 3. The controller invokes the service layer.
 4. The service creates the ViewModel (Presentation DTO).
 5. The controller places the ViewModel into Spring's Model.
 6. The controller returns the Thymeleaf template name.
 7. Thymeleaf merges the ViewModel with the HTML template. Thymeleaf never communicates directly with the database or the service layer, Thymeleaf only sees the constructed ViewModel
 8. The completed HTML page is returned to the browser.
- Students should understand that

Thymeleaf

Students should understand the role of Thymeleaf within Spring MVC.

1. Why Thymeleaf is a template engine.
2. Why Thymeleaf contains almost no business logic.
3. Why business rules belong in the service layer.
4. Why controllers prepare data for presentation.
5. Why Thymeleaf renders HTML rather than JSON.
6. Why REST APIs and Thymeleaf pages can coexist within the same Spring Boot application.

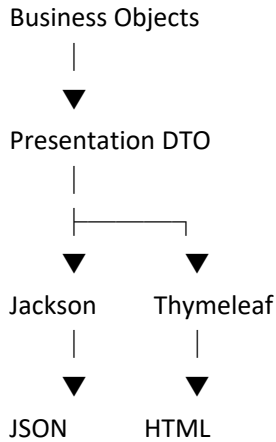
ViewModels

Students should understand the relationship between the application's business model and its presentation model.

1. Why MVC controllers use services prepare ViewModels.
2. Why a ViewModel contains only the data required by a specific page.
3. Why ViewModels should not expose JPA entities directly.
4. Why a ViewModel may contain data from multiple business objects.
5. Why different pages often require different ViewModels.

Presentation DTOs and ViewModels

REST controllers and MVC controllers perform the same architectural task. Both request the business layer to create a presentation DTO. The only difference is the rendering engine that consumes that DTO. In a REST controller, the Jackson library consumes the DTO, translating it to JSON. In a MVC controller, the Thymeleaf library consumes the ViewModel translating it to HTML via a template page.



Thymeleaf Binding

Students should understand how Thymeleaf connects HTML to the ViewModel.

Be able to explain the purpose of:

1. th:object
2. th:field
3. th:text
4. th:each
5. th:if
6. th:replace

Students should know which directives bind form fields, display values, repeat table rows, conditionally display content, and include reusable page fragments.

Login versus Authentication

Students should understand the difference between looking up credentials and authenticating a user. Students should understand that the Lab 4 login controller demonstrates credential lookup but does not provide secure authentication.